

1 STRUCTURE

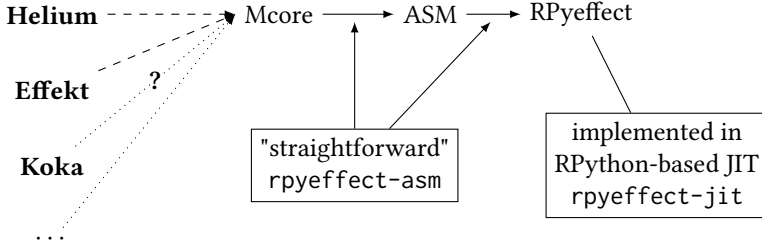


Fig. 1. Structure

2 MCORE

(actual format JSON-based or sexp)

Program Syntax:

Terms	$ \begin{aligned} t &::= x : T \\ & \lambda(\overrightarrow{x : T}). t \\ & t t \\ & \overrightarrow{t}; t \\ & \text{let } \overrightarrow{D} \text{ in } t \\ & \text{letrec } \overrightarrow{D} \text{ in } t \end{aligned} $	(typed) variables abstraction application sequencing
	<i>control:</i> $ \begin{aligned} & \text{fresh label} \\ & \text{reset } \Xi \text{ at } x : T \text{ with } (\overrightarrow{x : T}) t \text{ in } t \\ & \text{let } x : T = \text{shift } \Xi n \text{ in } t \\ & \text{let } x : T = \text{control } \Xi n \text{ in } t \\ & \text{resume } t (\overrightarrow{t}) \\ & \text{resumed } t \text{ in } t \end{aligned} $	generating labels reset/prompt shift control resume (with args) resume (with program that returns)
	<i>data, codata:</i> $ \begin{aligned} & \text{construct } C(\overrightarrow{t}) \\ & \text{project } t C f \\ & \text{match } t \{ \overrightarrow{C(\overrightarrow{x}) : t} \} \\ & \text{new } I \{ M(x : T) \Rightarrow t \} \\ & \text{invoke } t I.M(\overrightarrow{t}) \end{aligned} $	data type construction data type projection data type match codata/object construction method invocation
	<i>references:</i> $ \begin{aligned} & \text{letref } x : T = t \text{ at } t \text{ in } t \\ & \text{load } t \\ & \text{store } t t \end{aligned} $	allocation in region load from reference store in reference
Definitions	$D ::= x : T = t$	
Types	T	

Fig. 2. Syntax of MCore

3 ASM

(actual format similar or JSON-based)

Program Syntax:

Instructions	$s ::= \text{let } \overrightarrow{x} \leftarrow \overrightarrow{y}$ $ \quad \text{let } \text{const } x \leftarrow n$ $ \quad \overrightarrow{y} \leftarrow \text{primitive } N(\overrightarrow{x})$ <i>if0, switch, other constants ...</i> <i>control:</i> $ \quad \text{push } l(\overrightarrow{x})$ $ \quad \text{return } (\overrightarrow{x})$ $ \quad \text{jump } l(\overrightarrow{x})$ $ \quad y \leftarrow \text{shift } x_n x_p$ $ \quad y \leftarrow \text{control } x_n x_p$ $ \quad y, y_r \leftarrow \text{new stack } x_p l(\overrightarrow{x})$ $ \quad \text{push stack } x$ <i>data, codata:</i> $ \quad y \leftarrow \text{construct } C(\overrightarrow{x})$ $ \quad y \leftarrow \text{project } x C f$ $ \quad \text{match } x \{ \overrightarrow{C(\overrightarrow{y})} : l \}$ $ \quad y \leftarrow \text{new } I \{ \overrightarrow{M : l} \}(\overrightarrow{x})$ $ \quad \text{invoke } x M(\overrightarrow{x})$ <i>references:</i> <i>(new stack, see above)</i> $ \quad y \leftarrow \text{allocate } x_r(x)$ $ \quad y \leftarrow \text{load } x$ $ \quad \text{store } x x$	register movement constant integers primitive operations/"FFI" pushing a frame shift (w. label + offset) control (ditto) (meta)stack creation (with region and pro pushing a stack data type construction data type projection data type match codata/object creation method invokation allocation in region load from reference store in reference
--------------	---	--

Fig. 3. Syntax of ASM

4 RPYEFFECT

(actual format JSON-based)

Very similar to ASM. Most notable differences: Registers are indices, some restrictions on arguments for jumps etc, register movement in explicit small instructions.

Program Syntax:

Program	$P ::= \vec{B}_i$	programs
Blocks	$B ::= \{ \text{label}(\vec{x}_i) : s \}$ label	basic blocks block labels
Statements	$s ::= x \leftarrow v; s$ $\vec{x}_i \leftarrow \text{primitive } f(\vec{x}_j); s$	literals / constants primitive calls
<i>jumps</i>	jump B if x_1 then $B(\vec{x}_i)$ else s	jump conditional jump
<i>stack management</i>	push $B(\vec{x}_i); s$ return $(\vec{x}_i)$	push a frame return
<i>metastack management</i>	$x_1 \leftarrow \text{shift } x_2; s$ $x \leftarrow \text{new stack } B(\vec{x}_i); s$ push stack $x; s$	capturing stacks creating new stacks pushing a stack
<i>objects</i>	$x \leftarrow \text{new } V(\vec{x}_i); s$ $x.m(\vec{x}_i)$	object creation method invocation
<i>data</i>	$x \leftarrow \text{tag}(\vec{x}_i); s$ $x \text{ match } \{ \text{tag}(\vec{x}_i) \Rightarrow B \}$	data construction matching
<i>register management</i>	$x_1 \leftarrow x_2; s$ $x_1 \leftrightarrow x_2; s$ drop $x; s$	copying swapping dropping
VTables	$V ::= \{ \vec{m}_i \mapsto \vec{B}_i \}$	virtual table
Labels and tags	label, tag, m	
Variables	x	

Fig. 4. Syntax of the bytecode format .

REFERENCES